1. Searching Algorithms

Algorithm / Implementation	Best Case	Average Case	Worst Case	Notes
Binary Search (Array + Bubble Sort)	$O(N)$	$O(N^2)$	$O(N^2)$	The time is dominated by the $O(N^2)$ Bubble Sort. The search itself takes $O(\log N)$.
Binary Search on Linked List	$O(1)$	$O(N)$	$O(N)$	Finding the middle element requires linear traversal, neutralizing the $O(\log N)$ structural advantage.
Linear Search (Array)	$O(1)$	$O(N)$	$O(N)$	Best case occurs when the target is the first element.
Linear Search (Linked List)	$O(1)$	$O(N)$	$O(N)$	Sequential traversal node-by-node.

2. Linked Lists

Note: Time complexities depend heavily on the specific operation being performed (Search vs. Insert/Delete).

Structure	Operation	Best Case	Average Case	Worst Case

Singly Linked List	Access / Search	$O(1)$	$O(N)$	$O(N)$
	Insert / Delete (Head)	$O(1)$	$O(1)$	$O(1)$
	Insert / Delete (Tail)	$O(1)$	$O(N)$	$O(N)$
Doubly Linked List	Access / Search	$O(1)$	$O(N)$	$O(N)$
	Insert / Delete (Given Node)	$O(1)$	$O(1)$	$O(1)$
Circular Linked List	Access / Search	$O(1)$	$O(N)$	$O(N)$
	Insert / Delete (Head/Tail)	$O(1)$	$O(1)$	$O(1)$

3. Stacks

Implementation	Operation	Best Case	Average Case	Worst Case
Stack using Array	Push / Pop / Peek	$O(1)$	$O(1)$	$O(1)$*
Stack using Linked List	Push / Pop / Peek	$O(1)$	$O(1)$	$O(1)$

**If the array is dynamic and needs to resize upon pushing, the worst-case push is $O(N)$, but the amortized time remains $O(1)$.*

4. Queues

Implementation	Operation	Best Case	Average Case	Worst Case
Linear Queue (Array)	Enqueue / Dequeue	$O(1)$	$O(1)$	$O(1)$*
Linear Queue (Linked List)	Enqueue / Dequeue	$O(1)$	$O(1)$	$O(1)$
Circular Queue (Array)	Enqueue / Dequeue	$O(1)$	$O(1)$	$O(1)$
Circular Queue (Linked List)	Enqueue / Dequeue	$O(1)$	$O(1)$	$O(1)$
Deque (Array)	Insert / Delete (Both Ends)	$O(1)$	$O(1)$	$O(1)$
Deque (Linked List)	Insert / Delete (Both Ends)	$O(1)$	$O(1)$	$O(1)$

**Dequeue on a naive array implementation that shifts all elements down takes $O(N)$. A proper implementation using front/rear pointers is $O(1)$.*

5. Trees & Recursion

Algorithm / Structure	Best Case	Average Case	Worst Case	Notes
Tree (Standard Traversals)	$O(N)$	$O(N)$	$O(N)$	Preorder, Inorder, and Postorder must visit every node exactly once.
Factorial Tail Recursion	$O(N)$	$O(N)$	$O(N)$	Makes N recursive calls,

				passing the accumulated value.
Tower of Hanoi	$O(2^N)$	$O(2^N)$	$O(2^N)$	Requires exactly $2^N - 1$ moves to shift N disks.